

Demo: Loading Macros via include-in-header

This page demonstrates the `include-in-header` strategy for loading macros. The YAML front matter of this file includes:

```
format:
  html:
    include-in-header:
      - macros-header.html    # MathJax macro config script
  pdf:
    include-in-header:
      - macros.qmd            # raw LaTeX goes directly into preamble
  revealjs:
    output-file: demo-include-in-header-slides.html
    scrollable: true
    center: false
    include-in-header:
      - macros-header.html    # loader that reads macros.qmd
  docx: default
```

Note (HTML/RevealJS): `macros-header.html` is a small loader script (not a second macro-definition file). It reads `macros.qmd`, converts macro definitions to a MathJax config object, and sets `MathJax.tex.macros` before typesetting. For RevealJS, it also forces the built-in Reveal math plugin to use MathJax 3. This requires `macros.qmd` to be published as a static resource in the site output. [View the RevealJS slides output.](#)

Note (PDF): For PDF, `macros.qmd` can be included directly since its raw LaTeX goes into the document preamble.

Note: This demo uses `macros-header.html` / `macros.qmd` because both files are at the root of this repository. When using this repository as a Git submodule (cloned into a `macros/` subdirectory), replace with `macros/macros-header.html` and `macros/macros.qmd`.

The same configuration can be placed in a shared `_quarto.yml` to apply project-wide:

```
format:
  html:
    include-in-header:
      - macros/macros-header.html
  pdf:
    include-in-header:
      - macros/macros.qmd
  revealjs:
    include-in-header:
      - macros/macros-header.html
```

This inserts the macro definitions for each output format:

- **PDF:** `macros.qmd` goes into the LaTeX preamble via `include-in-header` — macros work, including `\providecommand` (note that LaTeX’s font-encoding initialization restores `\v` as a text caron accent at `\begin{document}`, so `\v{x}` produces a text accent error in math mode under `include-in-header` unless re-asserted in the preamble; use `\vec{x}` or `\vecf{x}` for robust cross-format vector notation).
- **HTML:** `macros-header.html` is inserted via `include-in-header` and loads macros from `macros.qmd` into `MathJax.tex.macros`.
- **RevealJS:** `macros-header.html` does the same loader step and switches the Reveal math plugin to MathJax 3, so slide math uses the same `macros.qmd` source.
- **docx:** Word documents do not use a LaTeX preamble or MathJax header, so `include-in-header` has no effect on math rendering for docx. The `docx: default` entry above produces docx output without macro support.

Example math

The table below uses macros defined with `\def` in `macros.qmd`, which work in all output formats that support MathJax or LaTeX:

Description	Code	Output
Greek: alpha	<code>\a</code>	α
Greek: beta	<code>\b</code>	β
Greek: gamma	<code>\g</code>	γ
Greek: lambda	<code>\l</code>	λ
Greek: sigma	<code>\s</code>	σ
Log-likelihood	<code>\llik</code>	ℓ
Independent	<code>X \ind Y</code>	$X \perp\!\!\!\perp Y$

Description	Code	Output
IID distributed	<code>X_i \siid \Normal(\m,</code> <code>\ss)</code>	$X_i \sim_{\text{iid}} N(\mu, \sigma^2)$
Vector: mu	<code>\vm</code>	$\tilde{\mu}$
Hat beta	<code>\hb</code>	$\hat{\beta}$

Color macros

`macros.qmd` defines a one-argument macro for each of the 19 [LaTeX base colors](#) (`\red`, `\blue`, `\green`, `\orange`, ...), using the `\color` switch form (`\color{red}{#1}`), which is supported by MathJax (HTML/RevealJS) and lualatex (PDF):

Description	Code	Output
Red	<code>\red{x}</code>	x
Blue	<code>\blue{y}</code>	y
Green	<code>\green{z}</code>	z
Orange	<code>\orange{\mu}</code>	μ
Combined	<code>\red{x} + \blue{y} =</code> <code>\green{z}</code>	$x + y = z$
Nested macros	<code>X_i \sim_{\red{\ind}} \blue{\Normal}(\orange{\mu_i},</code> <code>\green{\ss})</code>	$X_i \sim_{\text{ind}} N(\mu_i, \sigma^2)$

Note (docx): as described above, the `include-in-header` strategy does not load macros for docx output, so the color macros (like all other macros) are unavailable there. For docx, use the [include shortcode strategy](#) instead.